Development Prompt — NHL Hockey Analytics Platform Pre-v1 Consolidation Sprint

You are working on an existing NHL hockey analytics platform built with Flask, SQLAlchemy, SQLite, Plotly, and NHL API data.

Repository:

`https://github.com/david-turnbull/Hockey-game-analyzer`

The application already includes:

- NHL API ingestion
- raw NHL JSON caching
- normalized relational database
- teams, players, games, events, shots, and shifts
- game selector
- game overview
- scoring and penalty timelines
- interactive shot maps
- coordinate normalization
- Player Game Page
- shift-based TOI
- players-on-ice logic
- Corsi and Fenwick calculations
- forward-line combinations
- defence pairings
- prototype xG calculations
- data-quality validation
- automated tests
- real NHL fixture/regression tests

The application is already functional.

Do not redesign or rewrite the project.

The goal of this sprint is to make the existing analytics more defensible, reduce duplicated analytical logic, and prepare the repository for a clean portfolio-ready `v1.0`.

Main Objective

Complete the following three workstreams, in this order:

1. **Analytics correctness**
2. **Service architecture/refactoring**
3. **Portfolio and v1 release preparation**

Do not begin multi-season model training or build a new machine-learning xG model during this task.

Development Principles

Follow these rules throughout the work:

1. Inspect existing code before modifying it.
2. Preserve existing working functionality.
3. Make small, logical commits/changes.
4. Avoid unnecessary dependencies.
5. Reuse existing models and services where practical.
6. Prefer one authoritative implementation of an analytical concept.
7. Every correctness fix must receive a regression test.
8. Do not fabricate NHL data.
9. Preserve raw NHL source information.
10. Derived analytics must be reproducible.
11. Prefer explicit hockey definitions over assumptions.
12. Do not silently change the semantic meaning of existing fields.
13. Keep backward compatibility where reasonably possible.
14. Do not add major new UI features during this sprint.
15. Do not begin a trained xG model.

Before writing code, inspect the repository and produce a concise implementation plan showing:

- relevant files
- planned modifications
- database/schema implications
- new tests
- any migration requirements

Then implement the work incrementally.

Workstream 1 — Historical Player-Team Attribution

Problem

The existing `Player` model contains a field such as:

```
current_team_id
```

This is useful for representing a player's current team, but it is not reliable for historical game analysis.

For example:

```
October:
Player X plays for Calgary

March:
Player X is traded to Vancouver
```

If `current_team_id` later becomes Vancouver, querying the October game must still identify the player as a Calgary player.

Historical game attribution must not depend on the player's current team.

1.1 Introduce a Game-Player/Roster Relationship

Implement an authoritative per-game player-team relationship.

A model such as:

```
GamePlayer
-----------------------
game_id
player_id
team_id
position
```

is recommended.

Use naming consistent with the existing project.

Required properties:

- composite uniqueness for `(game_id, player_id)`
- foreign key to `Game`
- foreign key to `Player`
- foreign key to `Team`
- player position where available
- appropriate indexes

Do not duplicate unrelated player information.

The `Player` model should remain the identity/person record.

The new table represents:

> "Which team did this player represent in this specific game?"

1.2 Populate Game Rosters During Ingestion

Update the NHL ingestion/loading pipeline so the game-player association is created automatically.

Prefer explicit NHL roster/team information where available.

If shift records are used as supporting evidence, ensure the logic is deterministic.

The loader must remain idempotent.

Re-ingesting the same game must not create duplicate roster rows.

1.3 Remove Historical Dependence on `current_team_id`

Search the codebase for places where:

```
player.current_team_id
```

is used to determine historical game membership.

Replace those cases with the authoritative game-player relationship.

This is especially important for:

- Player Game Page
- player-game statistics
- possession statistics
- line combinations
- defensive pairings
- players-on-ice analysis
- future on-ice analytics

`current_team_id` may remain useful for current player metadata, but it must not determine historical game identity.

1.4 Historical Team Regression Tests

Add tests covering a traded-player scenario.

For example:

```
Game A:
Player 100 -> Team CGY

Later:
Player.current_team_id -> VAN

Historical query for Game A:
Player 100 must still be CGY
```

Test the actual service behaviour, not merely the database row.

Workstream 2 — Define True 5v5 Possession Metrics

Problem

The application currently uses an `EV` manpower classification for possession calculations.

Equal-strength situations may include:

```
5v5
4v4
3v3
```

However, analytics labelled:

```
5v5 Corsi
5v5 Fenwick
```

must include only actual 5-on-5 play.

2.1 Separate `EV` from `5v5`

Preserve the existing high-level manpower classification:

```
EV
PP
PK
SO
...
```

but introduce or use an explicit strength-state condition for true 5v5.

For example:

```
home_skaters = 5
away_skaters = 5
```

or:

```
team_strength_state == "5v5"
```

Use whichever representation is most authoritative in the existing schema.

2.2 Correct Corsi/Fenwick Calculations

Review:

```
calculate_possession_stats()
```

and related methods.

If the metric is presented as:

```
5v5 CF
5v5 CA
5v5 CF%
5v5 FF
5v5 FA
5v5 FF%
```

then only true 5v5 events should be included.

Do not include:

- 4v4
- 3v3
- power play
- penalty kill
- empty-net situations
- shootouts

unless specifically requested by a different analytical mode.

2.3 Preserve Future Extensibility

Structure the filtering logic so future views could support:

```
5v5
All Even Strength
Power Play
Penalty Kill
All Situations
```

Do not necessarily build the UI for all of these now.

The important requirement is that the underlying service does not hard-code ambiguous semantics.

2.4 Add Possession Regression Tests

At minimum test:

```
5v5 shot -> included

4v4 shot -> excluded from 5v5

3v3 shot -> excluded from 5v5

5v4 shot -> excluded

4v5 shot -> excluded

shootout attempt -> excluded
```

Also verify CF%, FF%, and denominators using exact expected values.

Workstream 3 — Standardize Shift-Time Semantics

Problem

Current players-on-ice logic may use inclusive shift boundaries such as:

```
start <= event_time <= end
```

and:

```
range(start, end + 1)
```

This can cause two adjacent shifts to overlap at the exact second where one ends and another begins.

Example:

```
Player A:
10:00 -> 10:40
```

```
Player B:
10:40 -> 11:15
```

At `10:40`, both players may currently be considered active.

3.1 Establish a Shift Interval Convention

Use half-open intervals:

```
[start, end)
```

Meaning:

```
start <= t < end
```

A player is:

- on the ice at `start`
- off the ice at `end`

Document this convention.

3.2 Apply the Convention Consistently

Search the project for all shift-based logic.

Update all relevant locations, including:

- players-on-ice detection
- Corsi/Fenwick attribution
- goal attribution
- line combinations
- defensive pairings
- player TOI where relevant
- future on-ice services

Do not leave different analytical modules using conflicting interval semantics.

3.3 Test Shift Change Boundaries

Add exact regression tests.

Example:

```
Player A = [600, 640)
Player B = [640, 675)
```

Expected:

```
t = 639 -> Player A
t = 640 -> Player B
```

Player A and Player B must not both be counted solely because their shifts touch.

Test:

- event one second before change
- event exactly at change
- event one second after change

Workstream 4 — Create an Authoritative On-Ice Service

Problem

The application now contains several features that require the same core question:

> Which players were on the ice for Team X at time T?

This logic should not be independently reimplemented across services.

4.1 Create `OnIceService`

Create a dedicated service such as:

```
app/services/on_ice_service.py
```

Naming may follow existing project conventions.

It should provide an authoritative method similar to:

```
get_players_on_ice(
    game_id,
    elapsed_seconds,
    team_id=None,
)
```

Return structured player information rather than merely IDs where practical.

For example:

```
[
    {
        "player_id": 123,
        "team_id": 20,
        "position": "C",
    },
    ...
]
```

Use an appropriate Python representation consistent with the repository.

4.2 Centralize Shift Validity Rules

The service must consistently handle:

- zero-duration shifts
- anomalous shifts
- goalie shifts
- missing players
- malformed timing
- period transitions
- half-open interval boundaries

Do not duplicate these rules in possession and line-combination services.

4.3 Add Team-Specific Helpers

Useful helper methods may include:

```
get_team_players_on_ice(...)
get_skaters_on_ice(...)
get_goalie_on_ice(...)
```

Only add helpers that are genuinely useful.

Avoid unnecessary abstraction.

4.4 Refactor Existing Analytics to Use `OnIceService`

Update:

- possession calculations
- player on-ice calculations
- goals-for/goals-against attribution
- line combinations
- defensive pairings

to use the centralized service wherever appropriate.

There should be one authoritative answer to:

```
Who was on the ice at time T?
```

4.5 Add On-Ice Tests

Test:

```
normal 5-player group
goalie + five skaters
shift substitution boundary
zero-duration shift ignored
```

```
anomalous shift ignored where required
home and away filtering
```

Where practical, include a regression test against real cached NHL shift data.

Workstream 5 — Refactor `GameService`

Problem

`game_service.py` has grown significantly and now handles several separate analytical responsibilities.

The goal is not to create an elaborate architecture.

The goal is to split clearly independent analytical domains.

5.1 Proposed Service Structure

Evaluate a structure similar to:

```
app/services/
│
├── game_service.py
├── player_game_service.py
├── possession_service.py
├── on_ice_service.py
├── line_service.py
└── xg_service.py
```

Do not force this exact structure if the current repository suggests a cleaner minimal alternative.

5.2 Responsibilities

`game_service.py`

Keep responsibilities such as:

- game overview
- team game summary

- scoring timeline
- penalties
- basic game-level calculations

`player_game_service.py`

Move:

- player game statistics
- individual events
- player TOI
- player shot information
- player game summary logic

`possession_service.py`

Move:

- CF
- CA
- CF%
- FF
- FA
- FF%
- possession filtering

`line_service.py`

Move:

- forward combinations
- defence pairings
- line TOI
- line GF/GA
- line SF/SA

`on_ice_service.py`

Own:

- players-at-time-T logic
- shift interval semantics
- active skater determination

`xg_service.py`

Keep xG calculations here.

Do not expand xG methodology during this sprint.

5.3 Avoid Circular Dependencies

Services should not recursively depend on each other without clear ownership.

Prefer:

```
LineService
    ↓
OnIceService
```

rather than:

```
GameService
↔ LineService
↔ PossessionService
↔ GameService
```

Shared low-level analytical primitives should live in the lowest appropriate service.

5.4 Preserve Routes

Do not redesign the external URL structure unless necessary.

Existing pages and API endpoints should continue to work.

Routes may call new services internally.

Workstream 6 — Clarify xG Status

Problem

The current xG implementation uses manually chosen logistic coefficients.

It should not be presented as a statistically trained NHL xG model.

6.1 Rename/Describe It Appropriately

Use terminology such as:

```
Prototype xG
Prototype Expected Goal Estimate
Experimental xG Estimate
```

where user-facing presentation or documentation could otherwise imply the model was trained and validated.

Do not remove the feature.

It is useful infrastructure.

6.2 Document the Formula

Document:

- logistic transformation
- distance coefficient
- angle coefficient
- shot-type adjustments
- manpower adjustments
- any other features

Clearly distinguish:

```
hand-selected coefficients
```

from:

```
coefficients learned from NHL historical outcomes
```

6.3 Preserve the Existing xG Tests

Existing tests verifying mathematical behaviour should remain.

Add tests if necessary to ensure the prototype behaves consistently.

Do not claim predictive accuracy.

6.4 Explicitly Defer Trained xG

Add a roadmap item for:

```
Historical NHL shot dataset
Feature engineering
Train/validation/test split
Baseline logistic regression
Calibration
Brier score
Log loss
ROC-AUC
Calibration curves
Alternative models
```

Do not implement these in this sprint.

Workstream 7 — Database and Schema Validation

After introducing `GamePlayer` and refactoring services, run the database integrity checks.

Verify:

```
No orphan GamePlayer records
No duplicate GamePlayer rows
Every roster player references a valid team
Every roster player references a valid game
Every roster player references a valid player
Shift team agrees with historical player-game team where appropriate
```

Do not automatically overwrite disagreements without investigating them.

Report mismatches.

Workstream 8 — Improve GitHub README

The current README undersells the application.

Rewrite it as a portfolio-quality project overview.

The README should include the following sections.

8.1 Project Title and Summary

Explain clearly that this is an NHL game-analysis platform built to explore:

- game events
- shot locations
- player performance
- shifts
- possession
- line combinations
- defensive pairings
- experimental xG

Keep the description professional.

Avoid marketing exaggeration.

8.2 Screenshots

Create a section with placeholders or actual repository-relative image links for:

```
Game selector
Game overview
Shot map
Player Game Page
Line combinations / possession
```

If screenshot files are already available, use them.

Otherwise document where they should be added.

Do not fabricate screenshots.

8.3 Architecture

Add an architecture/data-flow diagram using Mermaid if supported by GitHub Markdown.

For example:

```
NHL API
   ↓
Raw JSON Cache
   ↓
Transform / Validate
   ↓
SQLite / SQLAlchemy
   ↓
Service Layer
   ↓
Flask Routes / API
   ↓
Interactive Analytics UI
```

Include major services after the refactor.

8.4 Feature Summary

Document:

- ingestion
- reproducibility

- game overview
- spatial analytics
- player analysis
- shift analysis
- possession
- line combinations
- xG prototype

8.5 Analytical Definitions

Briefly explain:

```
Corsi
Fenwick
5v5
EV
PP
PK
TOI
xG
```

Do not assume all readers know hockey analytics terminology.

8.6 Data Source

Explain that the project uses NHL API data.

Describe:

- raw response preservation
- cached fixture use
- database transformation
- reproducibility

Do not imply NHL endorsement.

8.7 Testing

Describe:

- unit tests
- regression tests
- real NHL cached fixture tests
- database integrity validation

Include the test command.

8.8 Limitations

Be explicit.

Examples may include:

- NHL public data limitations
- second-level shift timing
- on-ice reconstruction assumptions
- prototype xG coefficients
- incomplete multi-season validation
- no production deployment yet

A limitations section strengthens the project.

8.9 Roadmap

Use a concise roadmap.

Example:

```
v1.0
- Core NHL ingestion
- Game analytics
- Player analytics
- Shift analysis
- Possession
- Line combinations

v1.x
- Full-season ingestion
```

```
- Performance optimization
- Expanded filtering

v2.0
- Multi-season shot dataset
- Trained xG model
- Model validation
- Advanced on-ice analytics
```

Adjust based on actual state.

Workstream 9 — Add Continuous Integration

If GitHub Actions is not already configured, create a minimal workflow:

```
.github/workflows/tests.yml
```

Run:

```
pytest
```

on:

```
push
pull_request
```

Use a supported Python version compatible with the project.

Install dependencies from the project's existing dependency file.

Do not create an unnecessarily complex CI pipeline.

The objective is:

```
GitHub commit
     ↓
Automated tests
     ↓
Pass / fail
```

Workstream 10 — Prepare v1.0 Release Criteria

Create a concise release checklist.

`v1.0` should not be tagged until the following are true:

- [ ] Fresh environment can install dependencies
- [ ] Database can be initialized from scratch
- [ ] At least one complete test game can be ingested
- [ ] Game selector works
- [ ] Game overview works
- [ ] Shot map works
- [ ] Player Game Page works
- [ ] Historical team attribution works
- [ ] True 5v5 possession filtering works
- [ ] Shift boundaries use a documented convention
- [ ] OnIceService is authoritative
- [ ] Line combinations work
- [ ] Defensive pairings work
- [ ] Prototype xG is clearly labelled
- [ ] Database integrity diagnostic passes
- [ ] Unit tests pass
- [ ] Real NHL fixture tests pass
- [ ] GitHub Actions passes
- [ ] README accurately describes current functionality
- [ ] Known limitations are documented

Do not automatically publish or tag a release unless explicitly instructed.

Prepare the repository so that it is ready to do so.

Workstream 11 — Clean-Environment Validation

At the end of implementation, test the application as though you are a new developer cloning the repository.

Use a fresh virtual environment if feasible.

Perform:

```
Clone/install dependencies
Initialize database
```

```
Run tests
Load representative NHL data
Start Flask application
Open major pages
```

Verify:

```
Season selector
Team selector
Game selector
Game overview
Shot map
Player Game Page
Possession metrics
Line combinations
Defence pairings
Prototype xG
```

Do not rely only on the developer's existing database.

Do Not Implement During This Sprint

Do not implement:

```
New trained xG model
XGBoost
Random forest xG
Neural-network xG
WAR
RAPM
Player projections
Game predictions
Betting models
Live game streaming
Video tracking
Computer vision
Multi-season model training
Major frontend redesign
User accounts
Cloud infrastructure
```

These are future milestones.

Required Regression Tests

At minimum ensure tests cover:

```
Historical player-team attribution
Traded player remains attached to historical team
GamePlayer loader is idempotent
5v5 event included in 5v5 Corsi
4v4 excluded from 5v5 Corsi
3v3 excluded from 5v5 Corsi
PP excluded from 5v5 Corsi
PK excluded from 5v5 Corsi
Shootout excluded from possession
Shift end boundary is exclusive
Replacement player begins exactly at shift start
No duplicate active player caused by touching shifts
OnIceService returns correct team
Zero-duration shifts do not create on-ice presence
Player Game Page still works after refactor
Line combinations still produce expected output
Defence pairings still produce expected output
Prototype xG behaviour remains unchanged
```

Prefer exact expected hockey results rather than vague assertions.

Definition of Done

This sprint is complete when:

- [ ] Historical game-team attribution no longer relies on `Player.current_team_id`
- [ ] `GamePlayer` or equivalent provides authoritative historical roster membership
- [ ] Possession metrics labelled 5v5 contain only true 5v5 events
- [ ] Shift intervals use `[start, end)` consistently
- [ ] One authoritative `OnIceService` exists
- [ ] Possession logic uses the authoritative on-ice implementation
- [ ] Line-combination logic uses the authoritative on-ice implementation
- [ ] `GameService` has been split into sensible analytical services
- [ ] No unnecessary circular dependencies exist
- [ ] Existing routes continue to work
- [ ] Prototype xG is accurately described
- [ ] xG methodology limitations are documented
- [ ] README reflects current project capabilities

- [ ] Architecture/data-flow documentation exists
- [ ] Limitations are documented
- [ ] GitHub Actions runs tests automatically
- [ ] All unit tests pass
- [ ] All NHL regression fixture tests pass
- [ ] Database integrity checks pass
- [ ] Clean-environment startup succeeds
- [ ] Repository is ready for a `v1.0` release

Final Deliverable

At completion, provide an engineering report with the following sections.

1. Files Changed

List each major changed file and explain why it changed.

2. Historical Roster Changes

Explain:

- new schema
- ingestion logic
- how historical team attribution works
- any migration implications

3. Possession Corrections

Explain the exact definition now used for:

```
5v5 Corsi
5v5 Fenwick
```

4. Shift Semantics

State explicitly that shifts use:

```
[start, end)
```

and explain why.

5. On-Ice Architecture

Explain the authoritative method for determining:

```
players on the ice at time T
```

and which analytics consume it.

6. Service Refactor

Show the final service structure and responsibilities.

7. Test Results

Report:

```
tests passed
tests failed
tests skipped
```

and summarize the new regression coverage.

8. Data Integrity Results

Report:

```
orphan records
duplicate game-player rows
team mismatches
invalid shifts
other warnings
```

using actual results.

9. README / Portfolio Improvements

Describe what was added.

10. Known Limitations

List remaining methodological or technical limitations.

11. v1.0 Readiness

Give one of:

```
READY FOR v1.0
READY WITH MINOR WARNINGS
NOT READY FOR v1.0
```

Explain the reasoning.

12. Recommended Next Milestone

Do not begin it, but recommend the next development phase.

The expected next milestone is:

Full-Season Scaling and Analytical Validation

This should include:

- complete NHL season ingestion
- ingestion/runtime profiling
- database performance testing
- data-integrity reporting across a season
- larger historical shot dataset
- preparation for a statistically fitted xG model

Do not implement this next milestone as part of the current task.